

Experiment – 8

Name: Arnav Prajapati

UID: 24BAI70131

Section: 24AIT_KRG G1

A. Experimental Procedure

Step 1: Implementation

Binary Search,

Subset Sum (Verification and Decision versions),

Traveling Salesman Problem (Brute Force).

The implementation must guarantee correctness, proper handling of edge cases, and readability.

Step 1: Implementation

The implementation step was concerned with creating three different algorithms whose behavior could be examined within the context of different complexity classes. Each of these three algorithms was implemented in order to achieve maximum readability and handling of edge cases.

Binary Search ($O(\log n)$)

- Approach: Implemented using iterative divide-and-conquer on sorted arrays
- Logic: Algorithm divides array into two halves by finding the mid-index each time and eliminates one of the halves that does not contain the target
- Edge Cases: Included empty arrays, one-element arrays, and cases where the target is less than the first element or greater than the last element.

Subset Sum (Verification and Decision)

- Decision Version ($O(2^n)$): Implemented using recursive function with memoization in order to find any subset that adds up to the target value (K)
- Verification Version ($O(n)$): Linear algorithm for verifying whether a given subset equals the target (K)

- Edge Cases: Included situations where the target value is zero, cases of negative integers if any, and sets of numbers where total of numbers is still less than K.

Traveling Salesman Problem (Brute Force - $O(n!)$)

- Approach: Permutation technique used in order to find the total distance between cities forming the Hamiltonian circuit
- Logic: The algorithm searches through all $(n-1)!$ permutations to find the one with minimum total distance
- Edge Cases: Small graphs, disconnected graphs, and weighted edges handled properly to validate logic regardless of graph density.

Step 2: Measure Metrics

Collect relevant metrics for each run of each program such as time taken in ms, number of operations, and feasibility status (Completed or Timeout). Organize the collected information for analysis purposes.

To measure the “Factorial Wall” and “Exponential Growth,” certain metrics were collected during execution

- Execution Time: High precision timer used to measure difference between sub-millisecond search results and several seconds of calculations using brute force techniques
- Number of Operations: Approximated using the Big-O theoretical complexity ($n!$ for TSP, 2^n for Subset Sum, and $\log n$ for Binary Search)
- Feasibility Status: Completed: Applied for executions where algorithm completed execution in under 30 seconds.
- Timeout: Applied for executions where combinatorial explosion prevented computer to complete computations (usually seen in TSP where $n > 15$).

B. Analysis Tasks (Critical Thinking)

Q1. Why Binary Search performs efficiently despite large input sizes

Ans. Binary Search is able to perform efficiently even for input sizes going up to millions due to its $O(\log n)$ theoretical complexity. It differs from Linear Search ($O(n)$) in that it eliminates half of the elements in each comparison. For example, increasing the number of inputs from 1,000 to 1,000,000 causes number of

operations go from roughly 10 to 20. This is a much slower growth compared to other algorithms, hence making it very scalable.

Q2. Why Subset Sum is difficult to solve but easy to verify

Ans. Subset Sum belongs to the class of NP-complete problems. In order to solve the decision problem, one needs to check all 2^n subsets. For $n = 100$, there will be more subsets than number of atoms in the observable Universe, therefore it is impossible to try out all combinations. Verifying solution is easy because you just need to perform a summing operation that has linear complexity of $O(n)$. This means that the proof itself is easy to verify but hard to generate.

Q3. Find the input size at which the TSP algorithm becomes infeasible

Ans. Threshold of infeasibility lies somewhere at $n=12$ or 15 for the standard brute-force algorithm. This is because complexity of the brute force algorithm is $O(n!)$. Though exponential function grows rapidly, factorial function grows even faster. At $n = 13$, we already have around 6.2 billion permutations. When $n=20$, the number of permutations ($2.4 * 10^{18}$) will take several decades to calculate on modern-day computers. Optimization is simply insufficient to counter the combinatorial explosion.

Q4. Difference between solving problem and verifying solution and give examples.

Ans. Problem solving is a technique of finding the solution in state space where the exact location of solution is unknown.

Example: Find subset with sum equal to K

Solution verification is a process of finding out whether a given solution meets all criteria of a problem

Example: Check if the subset sum equals K .

Q5. Why are NP-Complete problems considered the most difficult within NP class?

Ans. These problems are considered most difficult because:

There is no polynomial solution known

Solving one efficiently means solving others efficiently as well.

Requires checking an exponential number of possibilities

Examples: Subset Sum (Decision problem), Travelling Salesman Problem